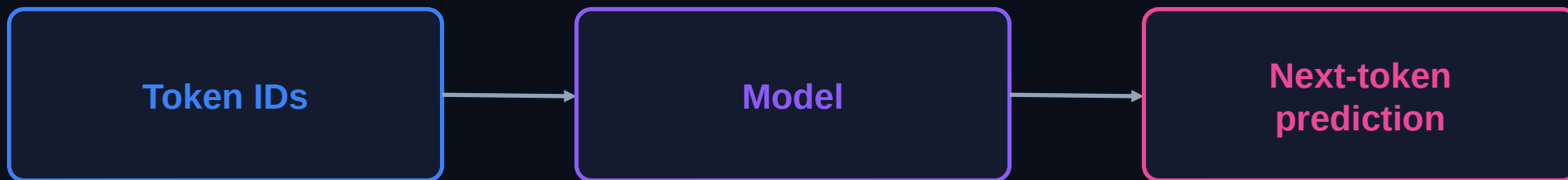


A FIRST STEP TOWARD A TRANSFORMER

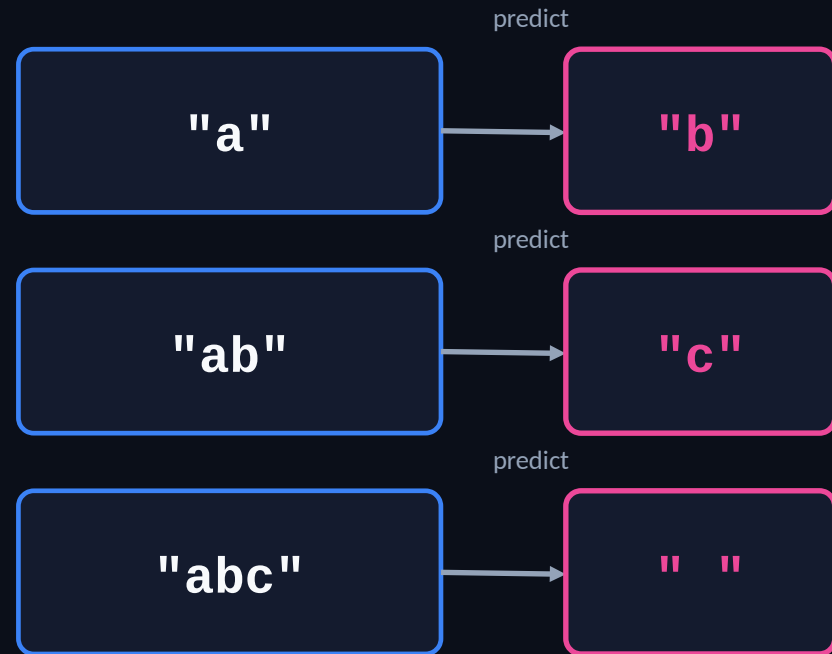
Inside Your Transformer Class

How token IDs become predictions, loss, and generated text



What Is a Language Model?

A language model predicts which token is likely to come next.



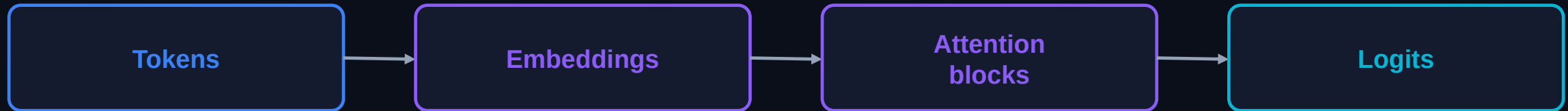
A token is a small piece of text. In this project, each character — including the space — is one token.

What Is a Transformer?

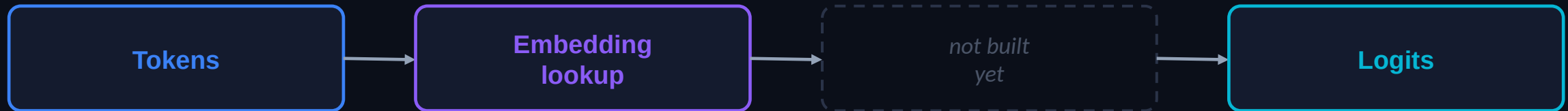
A Transformer is a neural network that predicts text by letting every token look at every other token in the sequence.

Compared with this class, right now:

A REAL TRANSFORMER



THIS CLASS, RIGHT NOW



The current class is a learning step toward a Transformer.
It does not contain attention yet.

The Pipeline, Term by Term

Four words from that pipeline you'll see again and again:

Tokens

A small piece of text. In this project, one character is one token.

Embeddings

A learned numeric representation of a token — a row of numbers standing in for it.

Attention blocks

Layers that let each token look at other tokens to decide what's relevant. Not in this class yet.

Logits

Raw prediction scores for what token should come next.

The next slides build each of these up from scratch, one at a time.

What Is an Embedding Table?

Each cell is a learned score for one possible next token.

current token	possible next token (one logit per column)				
	a	b	c	spc	.
a	•	•	•	•	•
b	•	•	•	•	•
c	•	•	•	•	•
spc	•	•	•	•	•
.	•	•	•	•	•

Used for: looking up one row is the entire forward pass — that row becomes the logits, the model's raw predictions for what comes next.

A real Transformer normally maps tokens to richer embedding vectors first; here, every row directly contains next-token scores.

What Are Logits?

Logits are raw prediction scores produced by a model.



A larger logit means the model favors that token more. They're raw scores, not probabilities — they can be negative and don't add up to 1.

Used for **Training**

Compared with the correct token (cross-entropy) to compute loss.

Used for **Generation**

Turned into probabilities (softmax), then sampled to pick the next token.

What Is PyTorch?

PyTorch is a Python library for creating, training, and running neural networks.

```
import torch
import torch.nn as nn
from torch.nn import functional as F
```

torch

Tensors and mathematical
operations

torch.nn

Neural-network building
blocks

F

Functions such as softmax
and cross-entropy

What Is nn.Module?

```
class Transformer(nn.Module):
```

nn.Module is the base class for PyTorch models.

nn.Module



↓

gives the class:

learned parameters

model calls

training support

device support

```
super().__init__()
```

Why this line is required:

`super().__init__()` runs `nn.Module`'s own setup first. That setup creates the internal bookkeeping PyTorch needs to track this model's layers and learned parameters.

Skipping it means the model can't find its own parameters later.

The Model's Setup: `__init__`

`__init__` creates the model's learned components once.

```
def __init__(self, vocab_size: int):  
    super().__init__()  
    self.token_embedding_table = nn.Embedding(  
        vocab_size,  
        vocab_size,  
    )
```

`vocab_size` is how many distinct tokens exist (5 in our example). `token_embedding_table` is the learned lookup table this model uses to make its predictions.

What Does forward Mean?

```
logits, loss = model(idx, targets)
```



```
logits, loss = model.forward(idx, targets)
```

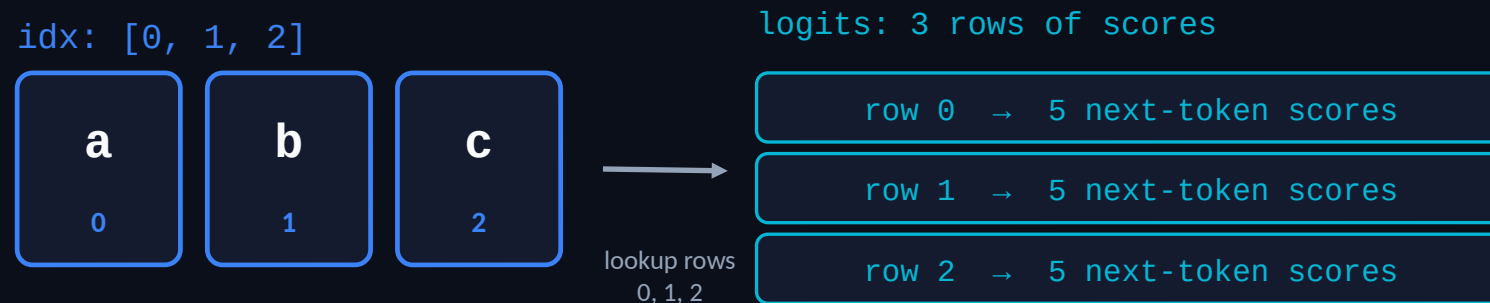
forward defines how input moves through the model.



In PyTorch, call `model(...)`, not `model.forward(...)` directly.

Step 1: Turn Token IDs into Logits

```
logits = self.token_embedding_table(idx)
```

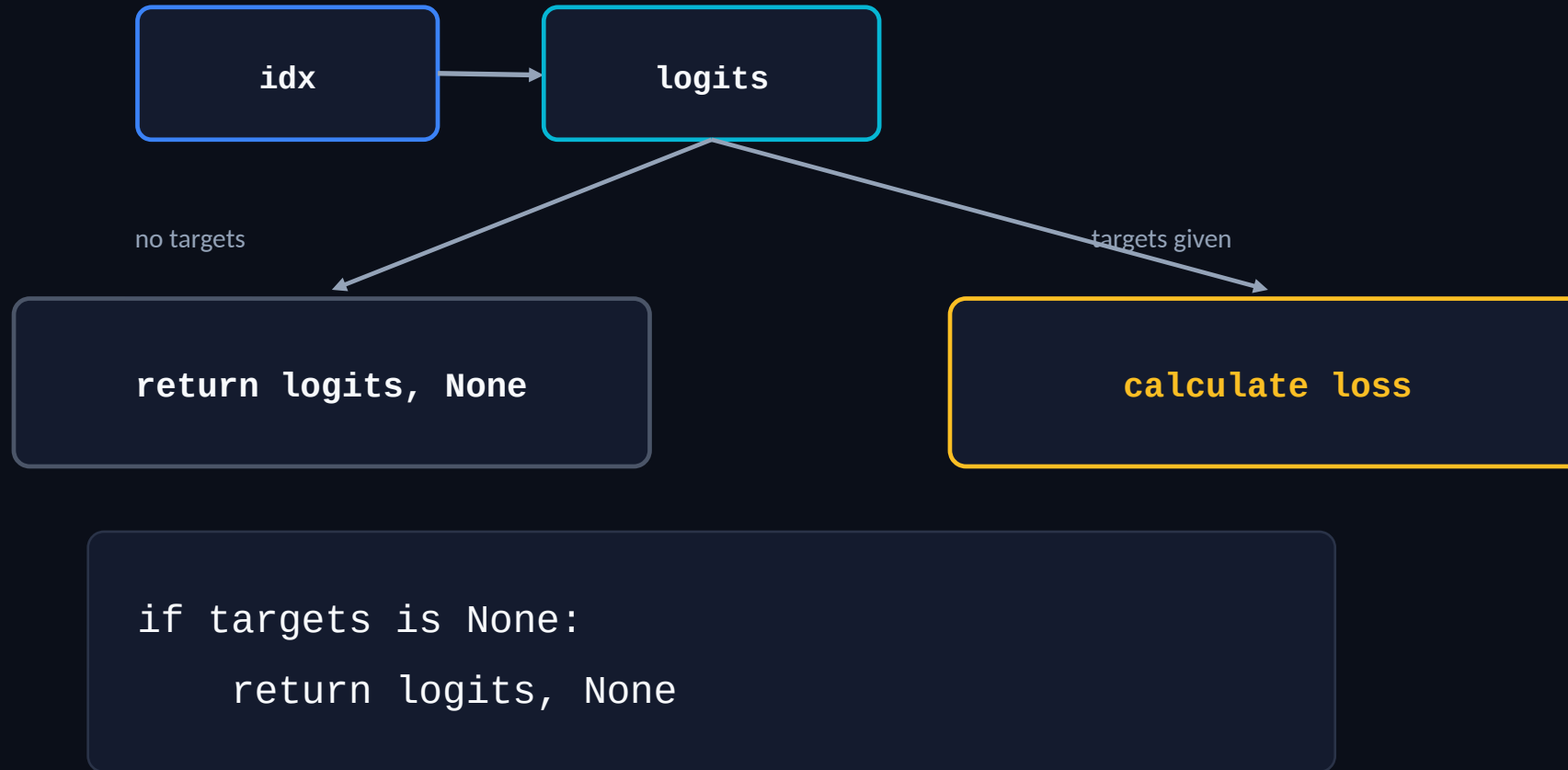


idx: (B, T)

logits: (B, T, C)

(1, 3) → (1, 3, 5)

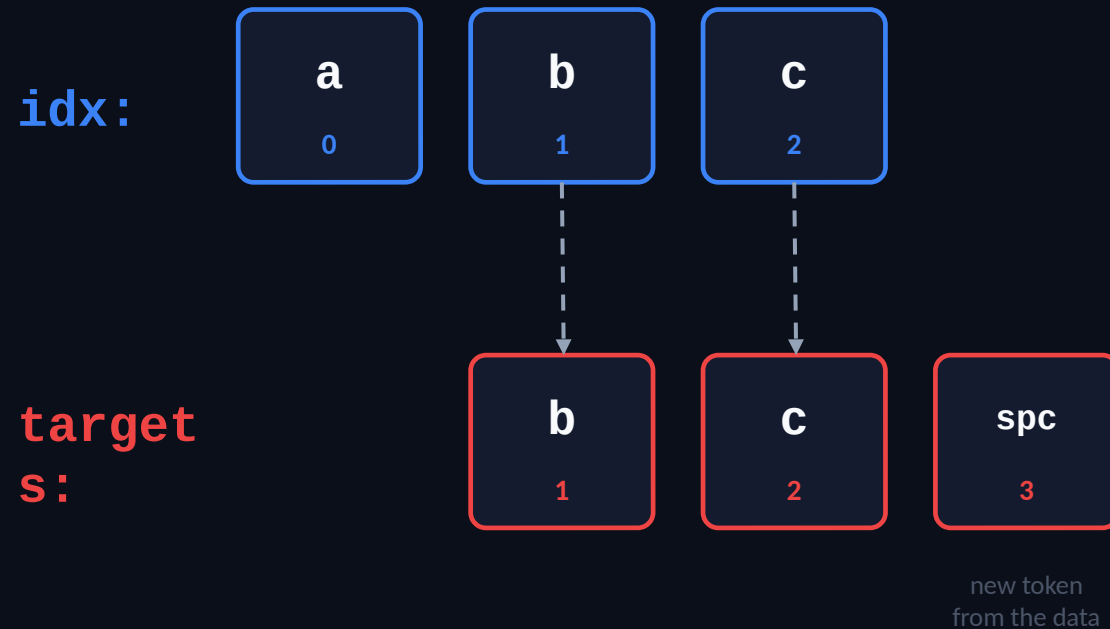
Why Does forward Accept Targets?



- Training supplies targets so the model can measure its mistakes.
- Generation has no known answer, so it does not calculate loss.

Inputs and Targets Are Shifted

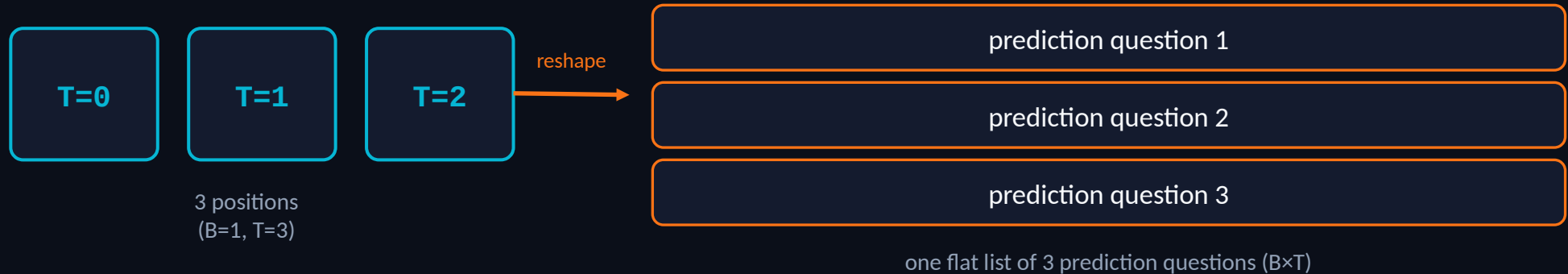
Input: $a \rightarrow b \rightarrow c$ Target: $b \rightarrow c \rightarrow \text{" "}$



At every position, the target is the token that appears next.

Why Flatten the Logits?

logits: (B, T, C) $\longrightarrow$ logits_for_loss: $(B \times T, C)$
targets: (B, T) targets_for_loss: $(B \times T)$



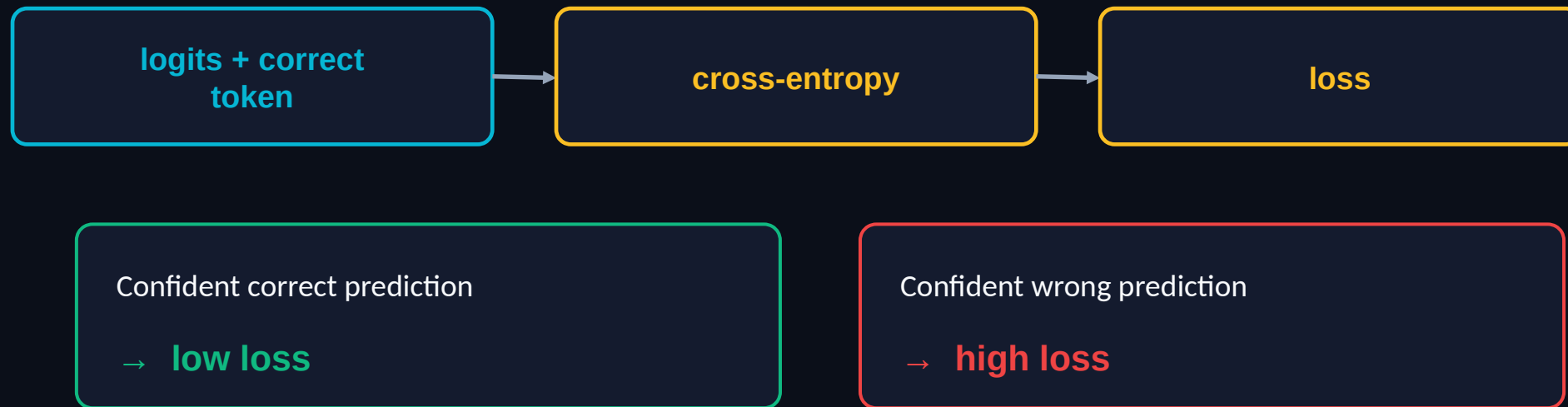
```
batch_size, sequence_length, vocab_size = logits.shape

logits_for_loss = logits.reshape(
    batch_size * sequence_length, vocab_size)
targets_for_loss = targets.reshape(
    batch_size * sequence_length)
```

What Is Cross-Entropy Loss?

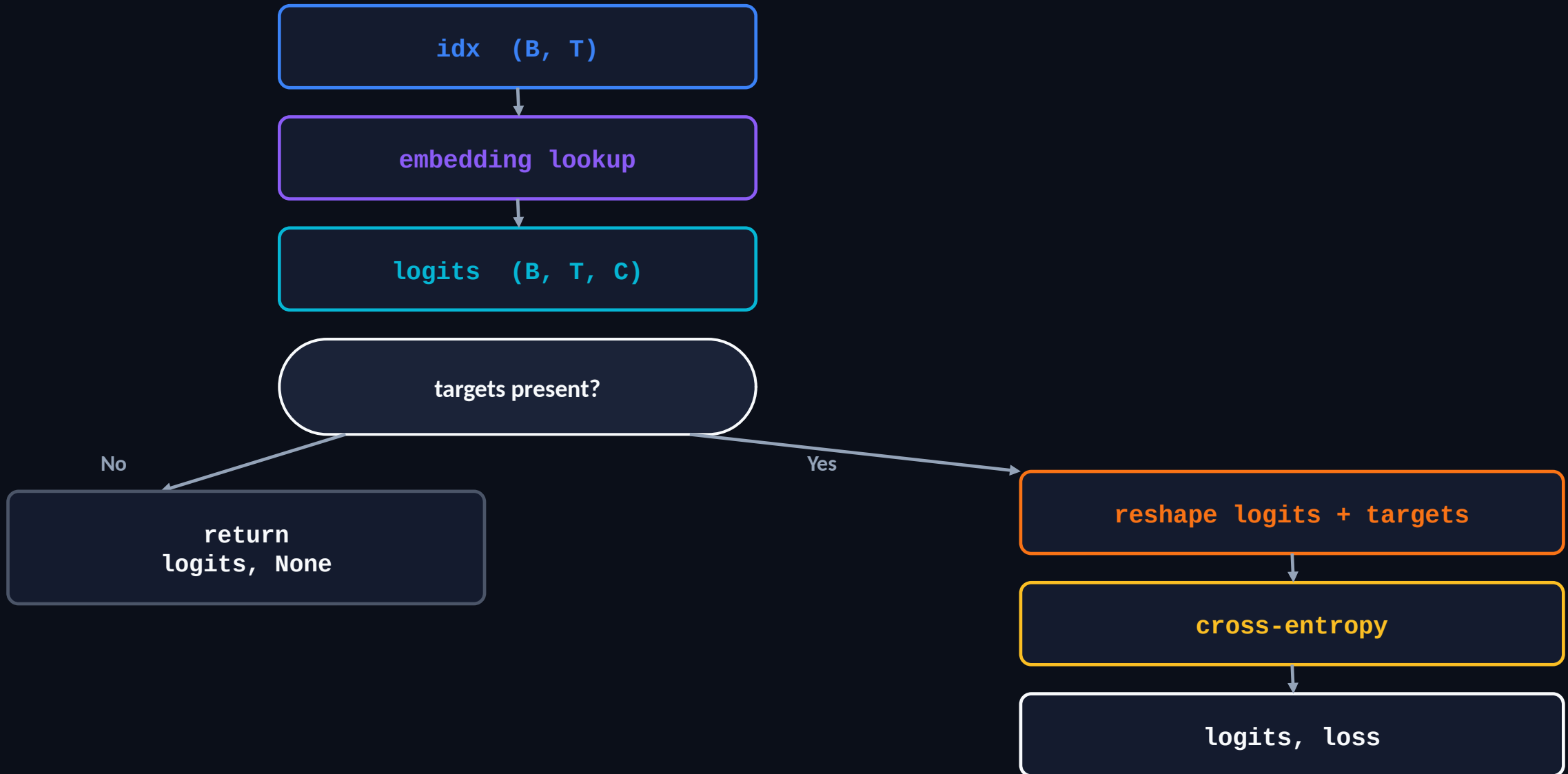
```
loss = F.cross_entropy(  
    logits_for_loss, targets_for_loss)
```

Loss is one number measuring how wrong the model's predictions are.



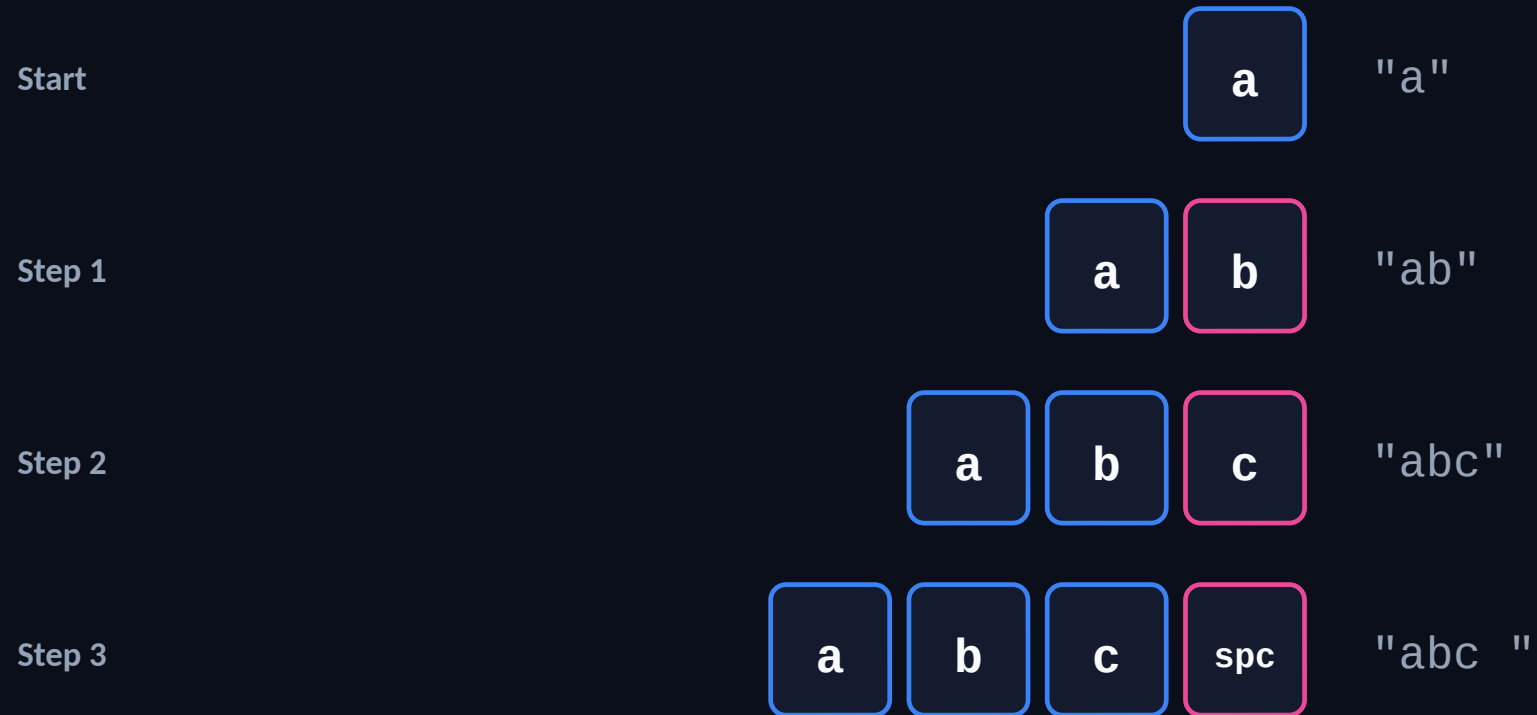
Cross-entropy compares the logits with the correct token ID. During training, PyTorch uses this loss to calculate how the embedding table should change.

The Complete forward Flow



What Does generate Do?

Generation repeatedly predicts and appends one new token.



`max_new_tokens` controls how many times this loop runs.

Generation Step 1: Run the Model

```
logits, _ = self(idx)
```

_ means the returned loss is intentionally ignored.



No targets are supplied during generation, so forward returns loss = None.

Generation Step 2: Use the Final Position

```
next_token_logits = logits[:, -1, :]
```

:

every sequence
in the batch

-1

the final
time position

:

every possible
next token

(B, T, C) → **(B, C)**

Earlier positions predicted earlier tokens. Only the last position predicts what should be appended now.

Generation Step 3: Softmax

```
next_token_probabilities = F.softmax(  
    next_token_logits, dim=-1)
```

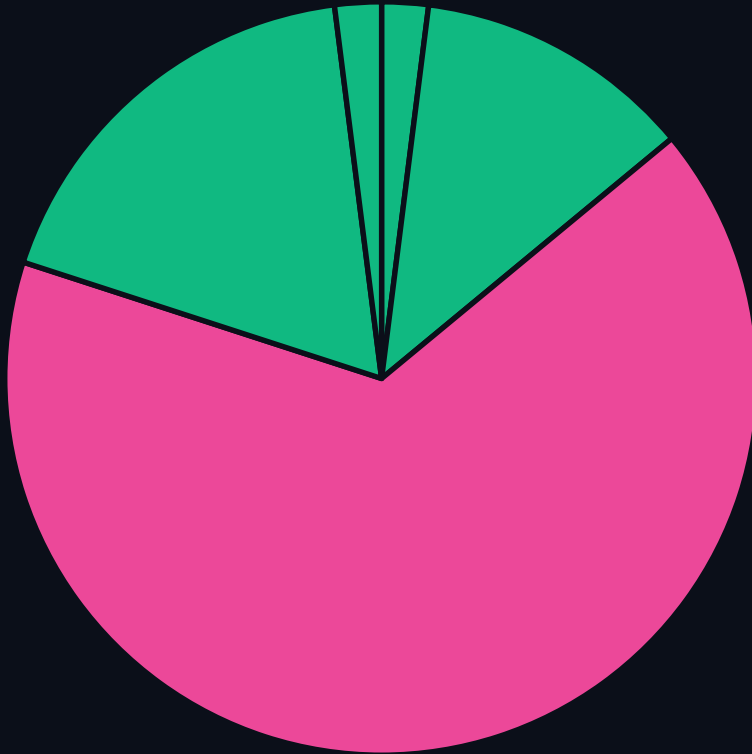


Softmax turns raw logits into probabilities that add up to 1.

`dim=-1` applies softmax across the final dimension: the vocabulary choices.

Generation Step 4: torch.multinomial

```
next_token = torch.multinomial(  
    next_token_probabilities, num_samples=1)
```



This spin landed on

"c"

`torch.multinomial` randomly samples an index using the supplied probabilities. High-probability tokens are more likely — but lower-probability ones stay possible.

a: 2% b: 12% c: 66% spc: 18% .: 2%

Sampling makes generated text variable. It does not always select the highest-probability token.

Generation Step 5: Append the Token

```
idx = torch.cat((idx, next_token), dim=1)
```

Current idx:



Next token:



Updated idx:



$(B, T) + (B, 1)$



$(B, T+1)$

dim=1 is the sequence (time) dimension — the new token is glued onto the end of each row.

The Complete Generation Loop



Training vs. Generation

Training

Input and **targets** are known

Calculates **cross-entropy loss**

Updates model **parameters**

Learns from correct answers

Generation

Only starting input is known

Does not calculate loss

Samples new tokens

Uses what it has learned

Important Limitation of This Model

Current model

next token depends only
on the current token

Full Transformer

next token can depend on
the whole context



A full Transformer uses attention to weigh which earlier words matter most for predicting what's next.

This class only ever looks at the single most recent token. A real Transformer would notice "cat" and "sat" earlier in the sentence — attention is how it decides which earlier tokens matter.

Vocabulary Cheat Sheet

Tensor

A multidimensional collection of numbers.

Token

A piece of text represented by an integer.

Embedding

A learned lookup table.

Parameter

A number the model learns during training.

Logit

A raw prediction score.

Probability

A normalized likelihood between 0 and 1.

Target

The correct answer.

Loss

A measurement of prediction error.

Cross-entropy

Compares logits with the correct token.

Softmax

Converts logits into probabilities.

Sampling

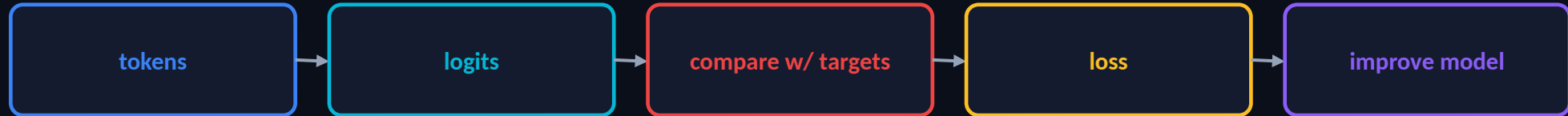
Randomly choosing by probability.

Batch

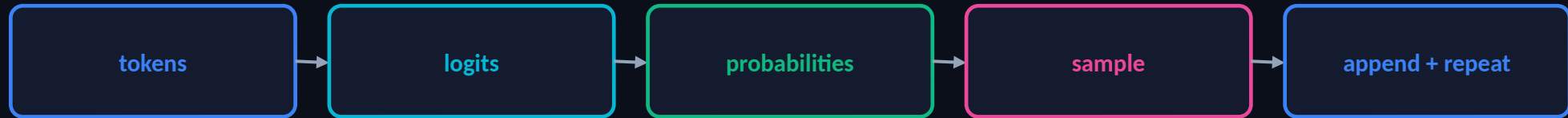
Multiple sequences processed together.

Final Mental Model

TRAINING



GENERATION



The model learns **scores** during **training** and uses those scores to **generate** tokens later.